pricetime

A stock exchange matching engine in C++20.

Built to be **proven correct** rather than claimed correct.

Marco Anthony Ayuste

THE PROBLEM

Every stock trade in the world passes through a matching engine.

It decides who trades with whom, at what price, in what order. It is simple to describe and **brutally unforgiving to build**, and when it breaks, it breaks expensively.

WHO EXPERIENCES IT

Everyone with a pension, an index fund, or a brokerage account, whether they know it or not.

WHY IT MATTERS

When it breaks, it breaks expensively.

- **Knight Capital, 2012.** A deployment reached 7 of 8 servers. The company lost **$460 million in 45 minutes** and ceased to exist independently.
- **Nasdaq, Facebook's IPO, 2012.** The opening auction entered an infinite loop because a validation step could never converge under load. **38,000 orders** were excluded. $10M penalty.
- **NYSE, fined $9M in March 2026.** Primary and backup systems ran at the same time, and **2,800+ opening auctions silently never happened**. There was no policy requiring anyone to check that they had.

THE GAP

Real matching engines are closed source. The public examples are toys.

- They use **floating-point prices**, which cannot represent money exactly.
- They are **never tested against real market data**.
- They publish **speed numbers with no methodology**.

So there is nothing a student, a regulator, or an engineer new to the domain can read, run, and check for themselves.

THE SOLUTION

Proven, not claimed.

CHECKED AGAINST ITSELF

The engine is written twice, once slow and obviously correct, once fast. A fuzz test drives **~200,000 operations** through both and demands byte-identical output after every single one.

GRADED BY THE VENUE

The feed says which orders executed. The engine reconstructs what must have happened and checks that it agrees. For Apple: **1,499 of 1,499**.

RUN ON REAL MARKET DATA

It decodes a live US exchange's actual historical feed, **42 million packets** and **50,163,616 orders** across 8,159 stocks, down to the individual order ID.

WATCHES FOR ABUSE

A surveillance layer screens the engine's own output for the patterns regulators care about, and reports what it measures without asserting intent.

IMPLEMENTATION

C++20. Zero dependencies. `g++` and `make`.

ENGINE

- Integer tick prices, never floating point
- Two-tier book keeps hot data inside the CPU cache
- Lock-free sharding across cores, proven identical to serial
- Multi-venue NBBO and trade-through detection

PROOF AND DELIVERY

- 45 tests, differential fuzz, CI on every push
- Clean under Address, UndefinedBehavior and ThreadSanitizer
- Compiled to a 42 KB WebAssembly module for the live demo
- An optional language model writes up findings, **never** on the execution path

RESULTS

The bottleneck was the CPU cache, not the algorithm.

	BEFORE	AFTER
Apple, real order flow	1,986 ns/order	50 ns/order
S&P 500 ETF, real flow	4,634 ns/order	62 ns/order
throughput	0.59 M/sec	12.8 M/sec

45
TESTS, ALL PASSING

100.0%
AGREEMENT WITH THE VENUE

~200k
FUZZED OPERATIONS

EXPECTED IMPACT

A reference anyone can check.

- An **open, MIT-licensed** matching engine in a category that is otherwise closed source, runnable end to end in under a minute.
- A **method others can copy**: differential testing against a second implementation, and replay graded against the venue's own execution records.
- A **surveillance layer** that measures real market behaviour and states what the evidence shows without asserting intent.
- A worked example of **where not to put a language model**: the component that decides who trades with whom must be deterministic and auditable.

`github.com/Minifigures/pricetime` · `pricetime-mu.vercel.app`